

Tema 10 - Intérpretes

Profesor Ramón López-Cózar Delgado, Curso 21-22

Función del Intérprete:

- Un intérprete toma un programa fuente y lo ejecuta inmediatamente, esta es la clave de la interpretación, la no existencia de un tiempo anterior donde se ha traducido.
- ¿Cuándo es útil un intérprete?
 - El programador trabaja en un entorno interactivo y se desean obtener los resultados de la ejecución de una instrucción antes de ejecutar la siguiente.
 - El programador lo ejecuta escasas ocasiones y el tiempo de ejecución no es importante.
 - Las instrucciones del lenguaje tienen una estructura simple y pueden ser analizadas fácilmente.
 - Cada instrucción será ejecutada una sola vez.
- ¿Cuándo no es útil?
 - Si las instrucciones del lenguaje son complejas.
 - Los programas van a trabajar en modo de producción y la velocidad es importante.
 - Las instrucciones serán ejecutadas con frecuencia.

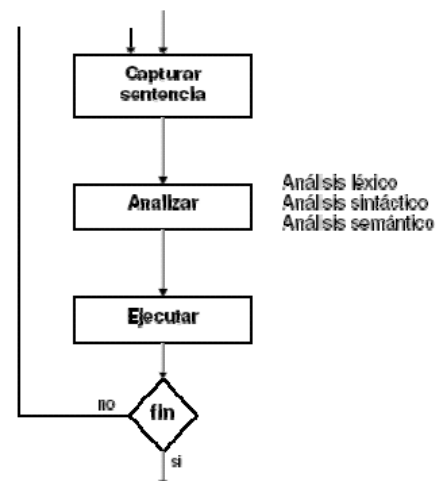
Tipos de Intérpretes:

- **Interpretación iterativa**
 1. Comienzo
 2. Repetir hasta última instrucción:
 - i. Capturar instrucción
 - ii. Analizar la instrucción
 - iii. Ejecución
 3. Fin
- **Interpretación recursiva**
 1. Comienzo
 2. Captura y análisis de la instrucción
 3. Ejecución
 - i. Si la instrucción es compuesta: Ejecución recursiva del intérprete
 - ii. Si la instrucción es simple: Ejecutar la instrucción
 4. Fin

Intérpretes iterativos:

- Se realiza cuando todas las instrucciones del lenguaje fuente son primitivas, es decir, no son sentencias compuestas o estructuradas.

- * Comienzo
- * Repetir:
 - * Capturar instrucción
 - * Analizar la instrucción
 - * Ejecutarla
- * Hasta la última instrucción
- * Fin



Ejemplo de un intérprete iterativo para un lenguaje de órdenes

Descripción del lenguaje de órdenes:

```
script ::= command*
command ::= command_name argument* end_of_line
argument ::= file_name
           | literal
command_name ::= create name
              | delete
              | edit
              | listfiles
              | quit
              | file_name
```

Secuencia de órdenes:

```
delete f1 f2
create f3 100
listfiles
edit f3
sort f3
quit
```

Código del interprete:

```
/* inicio */

/* Inicialización ... */
status := running ;
do {
    /* Captar la siguiente sentencia y analizarla ... */
    fetchAndAnalyze (com) ;
    /* ejecuta la instrucción ... */
    if (com.name == 'create')
        create (com.args[1], com.args[2]) ;
    else if (com.name == 'delete')
        delete (com.args) ;
    else if (com.name == 'edit')
        edit (com.args[1]) ;
    else if (com.name == 'listfiles')
        listFiles () ;
    else /* programa ejecutable */
        run (com.name, com.args) ;
} while (status != running) ; /* hasta que finalice la ejecución */

/* fin */
```

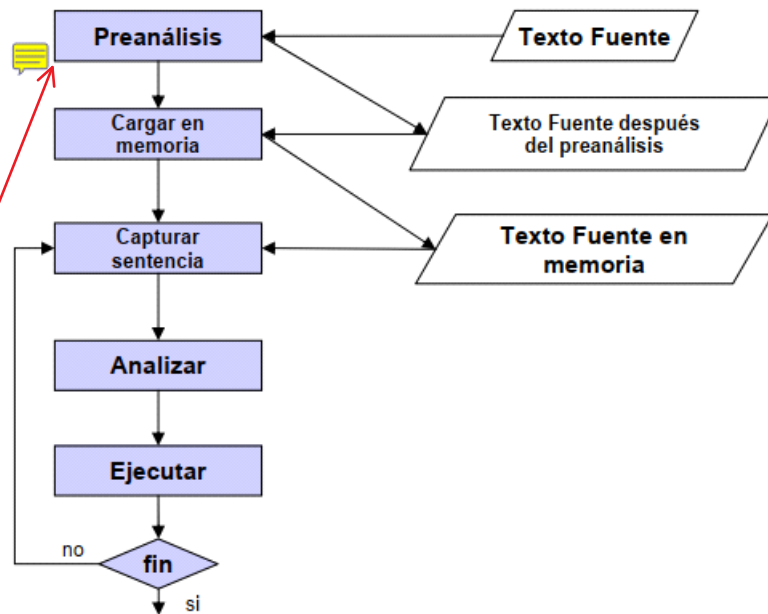
```
typedef char token[10] ;
typedef token tokensequence[8] ;
typedef struct {
    token name ;
    tokensequence args ;
} command ;
```

```
void fetchAndAnalyze (command com) ; /* Lee y analiza la siguiente sentencia */
void create (token fname, token size) ; /* Crea un archivo según el nombre y tamaño
                                         dado por argumento */
void delete (tokensequence fnames) ; /* Borra los archivos especificados por su nombre */
void edit (token fname) ; /* Edita el archivo especificado por su nombre */
void listFiles (void) ; /* Lista los nombres de todos los archivos propios
                        del usuario actual */
void run (token name, tokensequence args) ; /* Ejecuta el programa en código máquina que contiene
                                             la propia instrucción con los argumentos aportados
                                             en ella */

typedef enum status {running, halted, failed} ;
command com ;
```

Intérprete Iterativo para un Lenguaje de Alto Nivel Sencillo

- El primer paso para la construcción del intérprete es diseñar la máquina abstracta definida por el lenguaje, una especie de máquina virtual con registros para el programa.



- Se realiza el preanálisis del texto fuente para hacerlo más eficiente
- Se reparten las tareas de la fase de análisis

Preanálisis	Análisis
-----	léxico, sintáctico y semántico
léxico	sintáctico y semántico
léxico, sintáctico y semántico	-----

- Esta distribución puede variar de intérprete a intérprete, algunos pueden no tener preanálisis en general.

Compromiso entre la edición directa y eficiencia

- Los distintos formatos de las instrucciones en memoria son:
 - **Como texto:** Esta opción implica que cada vez que una instrucción es capturada se ha de realizar un análisis léxico, sintáctico (y semántico si es necesario). La carga y la edición es fácil.
 - **Como una secuencia de símbolos (Componentes léxicos):** Se realiza un análisis de léxico de las instrucciones antes de ser cargadas en memoria y serán analizadas sintáctica y semánticamente cuando son capturadas para ser ejecutadas. La carga y edición son relativamente fácil de realizar.
 - **Como un árbol sintáctico:** En este caso, la edición es difícil, por otro lado las instrucciones deben de ser analizadas léxica y sintácticamente cuando son cargadas, pero estarán preparadas para una ejecución inmediata.

DESCRIPCIÓN DE LENGUAJE (MINI BASIC)

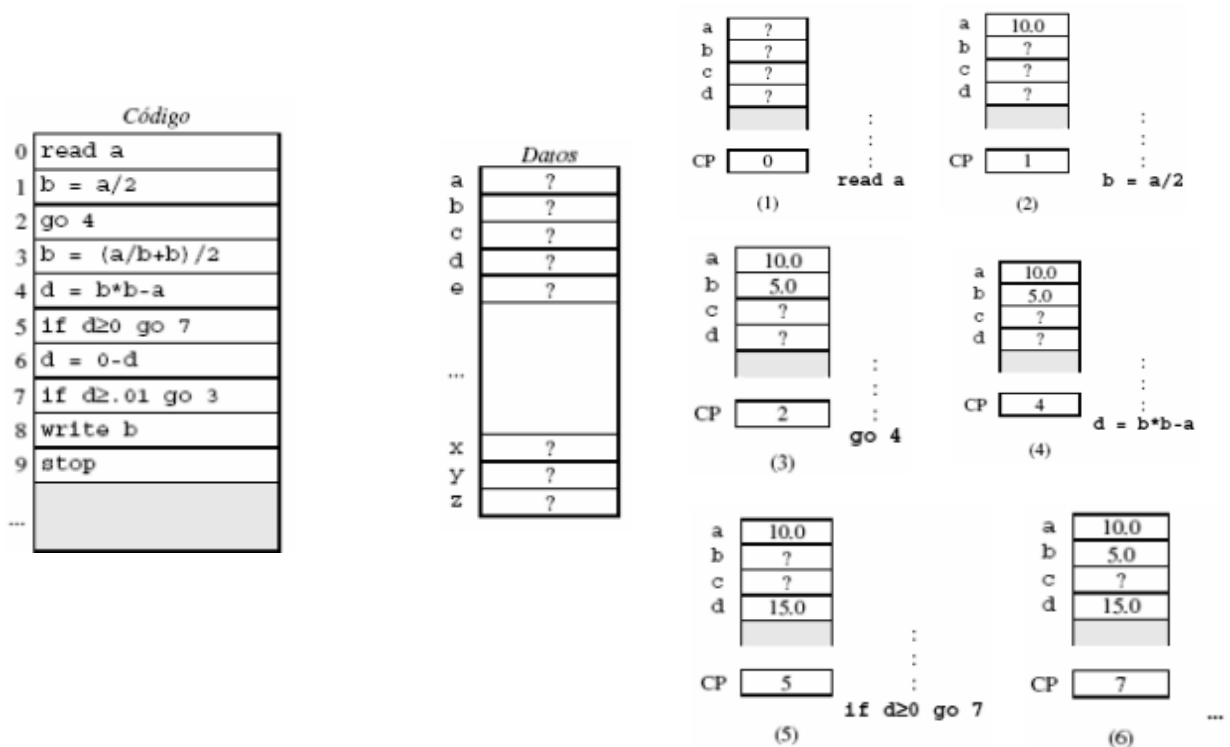
```

program ::= command *
command ::= variable = expression
          | read variable
          | write variable
          | go label
          | if expression relational_op expression go label
expression ::= primary_expression
            | expression arithmetic_op primary_expression
primary_expression ::= numeral
                  | variable
                  | ( expression )
arithmetic_op ::= '+' | '-' | '*' | '/'
relational_op ::= '=' | '<' | '>' | '!=' | '<=' | '>='
numeral ::= digito +
digito ::= 0|1|...|9
variable ::= albl...lz

```

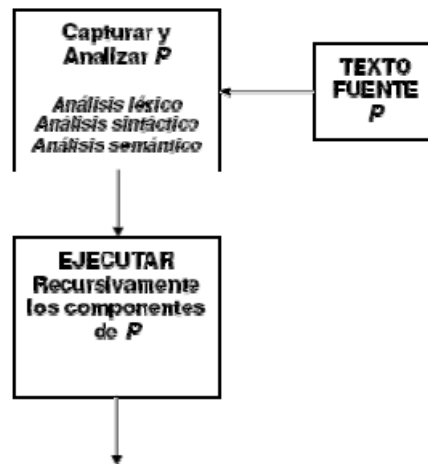
Interpretación iterativa de un lenguaje de alto nivel sencillo

- El primer paso para la construcción del intérprete es diseñar la máquina abstracta definida por el lenguaje.
- En el ejemplo se utiliza la siguiente máquina:
 1. El programa es cargado en una zona de memoria dividida en celdas, cada celda contiene una instrucción.
 2. El contador de programa *CP*
 3. Los datos están almacenados en 26 celdas, una para cada variable.
- Sobre las instrucciones se diferencian
 1. Su carga de memoria.
 2. Su captura de memoria para ser ejecutadas.



Intérpretes recursivos:

- Es necesario si el lenguaje fuente tiene funciones o sentencias compuestas, ahora se descomponen esas sentencias compuestas en simples recursivamente.
- El esquema del intérprete es el siguiente:
 - Comienzo
 - Capturar y Analizar C
 - En esta parte se ha de realizar el análisis de C (léxico, sintáctico y semántico), esto conlleva a realizar el análisis de las partes de C .
 - Ejecutar C
 - Operaría sobre la representación de C obtenida en el paso anterior, por ejemplo, de árbol sintáctico y ejecutaría recursivamente las subinstrucciones que componen C .
 - Fin



► El resto de los temas no fueron impartidos en este curso.